Week 10

pandas로 데이터 불러오기와 탐색

AI활용코딩연습(ZBE135)

이영호 교수 · 국립목포대학교 컴퓨터학부

지난주 돌아보기

```
scores.mean(axis=1)                  # 학생별 (행마다)
total[total >= 80]                   # 마스크
np.nanmean(avg_t)                    # 결측 무시
np.where(total >= 70, "통과", "재수강")
```

NumPy는 빠르지만 **열 이름이 없고 한 가지 자료형**만 → 표 데이터에는 **pandas**

오늘의 목표

1. `Series`와 `DataFrame`의 구조를 이해한다
2. `read_csv` → `head` · `info` · `describe` · `value_counts`로 **구조와 품질** 탐색
3. 열 선택, `loc` / `iloc`, **조건 필터**, 정렬, 새 열
4. 8주차 중간 실기를 **pandas로 다시** 하고 결과를 대조한다
5. AI 코드의 **`and`** **오류**와 **체인 인덱싱** 문제를 찾는다

수업 흐름

구간	시간	활동
도입 · 개념	20분	DataFrame 구조, 탐색 순서
안내 실습	45분	읽기 · 탐색 · 선택 · 필터 · 정렬 · 새 열
독립 · 짝 실습	35분	성적 데이터 탐색 보고 (질문 5개)
공유 · 점검	20분	pandas 함정 2가지, 과제 안내

이번 주 Colab 실습 열기

Series와 DataFrame

구조	비유	구성
`Series`	이름표 붙은 1차원 배열	값 + **인덱스**
`DataFrame`	엑셀 시트	같은 인덱스를 공유하는 **열(Series)들**

```
s = pd.Series([78, 92, 65], index=["김민준", "이서연", "박지호"])
s["이서연"], s.mean(), s[s >= 70]

df = pd.DataFrame({"name": [...], "midterm": [...], "final": [...]})
df.shape, df.columns, df.dtypes
```

데이터 읽기

```
df = pd.read_csv("students.csv", dtype={"student_id": str})
```

- 학번 · 전화번호 · 우편번호처럼 **계산하지 않는 숫자**는 문자열로
- 그렇지 않으면 `"01012345678"` → `1012345678` (앞자리 0 사라짐)

옵션	쓰임
`encoding="cp949"`	한글 깨짐 (Windows 엑셀 CSV)
`na_values=["-", "없음"]`	결측으로 볼 값 추가
`parse_dates=["date"]`	날짜 열 변환

첫 탐색 5종 세트

함수	알 수 있는 것
`df.head()` / `sample(3)`	실제 값의 모양
`df.info()`	행 수, 열 자료형, **Non-Null 개수**
`df.describe()`	숫자 열 요약 (평균 · 사분위 · 최소 · 최대)
`df.isna().sum()`	열별 결측 개수
`df["열"].value_counts()`	범주별 개수

> `assignment` → `float64`, Non-Null **77** → 결측 3개 (결측이 있으면 정수 열도 실수가 된다)

열과 행 선택

```
df["final"]                          # 열 하나 → Series
df[["name", "midterm", "final"]]     # 여러 열 → DataFrame (대괄호 두 겹)
```

표현	의미
`df.iloc[0]`	0번째 **위치**의 행
`df.iloc[:3, :2]`	앞 3행, 앞 2열
`df.loc[5, "name"]`	인덱스 **이름** 5인 행의 name
`df.loc[df["year"] == 1, ["name", "final"]]`	조건 행 + 특정 열

조건 필터

```
cond = (df["department"] == "컴퓨터공학과") & (df["final"] >= 85)
df[cond]

df[df["department"].isin(["간호학과", "경영학과"])]
df[df["final"].between(70, 79)]          # 70 이상 79 이하
df[df["assignment"].isna()]
df.query("year >= 3 and attendance < 12")
```

`&` `|` `~` + **괄호** (NumPy와 같다) · `query` 문자열 안에서는 `and` 가능

정렬

```
df.sort_values(["final", "student_id"], ascending=[False, True])
df.nlargest(3, "midterm")
df["year"].value_counts().sort_index()
```

- 여러 기준: 리스트로, 방향도 리스트로
- 동점 처리 기준을 **반드시** 정한다 (4주차 `key=(-점수, 이름)`과 같은 생각)

새 열 만들기

```
df["total"] = (df["midterm"] * 0.3 + df["final"] * 0.4
               + df["assignment"].fillna(0) * 0.3).round(1)

df["grade"] = pd.cut(df["total"], bins=[-1, 60, 70, 80, 90, 101],
                     right=False, labels=["F", "D", "C", "B", "A"])
df.loc[df["attendance"] < 12, "grade"] = "F"
```

- 열 연산은 **모든 행에 한 번에** (벡터화)
- `right=False` → `[60, 70)` 왼쪽 포함 → **90.0은 A**
- 조건부 수정은 `loc[행조건, 열] = 값`

8주차 실기 vs pandas

8주차 (순수 Python)	10주차 (pandas)
`csv.DictReader` + 형 변환 + 검증	`pd.read_csv`
`total_score` 함수 + 반복	열 연산 한 줄
`distribution` 딕셔너리	`value_counts()`
`dept_summary` 그룹 묶기	`groupby().agg()`

결과: **A 8 / B 22 / C 23 / D 12 / F 15** — 같다 ✅

> 원리를 알고 나서 도구를 쓰면 **결과를 검증할 수 있다**

실습 1 · 2 (Level 1 → Level 2)

Level 1 따라하기

- Series · DataFrame 만들기, `read_csv`, 탐색 5종 세트
- 선택 · `loc` / `iloc` · 필터 · 정렬 · 새 열

Level 2 바꾸기

- 정보보안학과 & 출석 15회 이상
- 총점 내림차순 · 이름 오름차순 상위 5명
- `improved = final - midterm` 가장 많이 오른 3명

실습 3 (Level 3): 탐색 보고

질문	방법
Q1. 몇 명 · 몇 열 · 결측은?	`shape`, `isna().sum()`
Q2. 등급 분포 = 8주차 결과?	`value_counts`, `assert`
Q3. 학과별 평균 · 통과율?	`groupby` 맛보기
Q4. 출석과 총점의 관계?	`corr`, 출석 구간별 평균
Q5. 과제 미제출의 영향?	필터, 결측 처리 비교

탐색은 **질문에서 출발**한다

groupby 맛보기

```
df["passed"] = df["grade"] != "F"
df.groupby("department").agg(
    count=("name", "size"),
    avg_total=("total", "mean"),
    pass_rate=("passed", "mean"),
)
```

`(새 열 이름) = (원래 열, 집계 함수)` — 자세한 것은 11주차

상관은 인과가 아니다

```
df["attendance"].corr(df["total"])     # 약 0.05
df.groupby(pd.cut(df["attendance"], [0, 11, 13, 15, 16]), observed=True)["total"].mean()
```

- 상관계수: −1 ~ 1, 0에 가까우면 선형 관계가 약함
- 상관이 높아도 **원인 → 결과**라고 말할 수 없다
- AI가 "출석이 성적을 올린다"고 쓰면 → **근거를 다시 확인**

결측 처리가 결론을 바꾼다

```
no_assign = df[df["assignment"].isna()]          # 3명
# 0점 처리 vs 과제 평균으로 채우기 → 총점 약 24점 차이
```

처리 방법	의미
0점	"미제출 = 0점" 규정이 있을 때
평균으로 채우기	측정 누락일 때 (성적에는 부적절)
행 제외	분석에서 빼도 될 때

> 결측 처리 방법은 **분석 목적과 규정**으로 정하고, 보고서에 밝힌다

AI 함정 1: and

```
df[df["final"] > 80 and df["year"] >= 3]
# ValueError: The truth value of a Series is ambiguous ...

df[(df["final"] > 80) & (df["year"] >= 3)]       # OK
```

`and` 는 **값 하나**의 참/거짓만 판단 → Series 전체에는 `&`

AI 함정 2: 체인 인덱싱

"1학년에게 보너스 5점 열을 만들어 줘"

```
df["bonus"] = 0
df[df["year"] == 1]["bonus"] = 5      # AI 코드
df.loc[df["year"] == 1, "bonus"].sum()   # 0 ← 안 바뀜!
```

`df[조건]`이 **복사본**을 만들고 → 복사본을 바꾼 뒤 버려짐

```
df.loc[df["year"] == 1, "bonus"] = 5    # 조건과 열을 한 번에
```

AI의 pandas 코드 체크리스트

확인	방법
조건에 `and` / `or`	`&`, `|`, 괄호
`df[...][...] = 값`	`df.loc[행, 열] = 값`
결과 행 수	`len()`, `shape`로 바로 확인
결측 처리	`isna().sum()`, `fillna` 여부
원본 변경	함수는 `df.copy()`로 시작
`count()` vs `size()`	결측 제외 vs 전체 행

이번 주 과제 (10점)

문제	내용	배점
1	`overview(df)` 행·열·결측·숫자 열·범주 개수	2
2	`find_students(df)` 조건 필터 + 다중 정렬	2
3	`add_total_grade(df)` · `top_names` (원본 불변)	3
4	`dept_report(df)` 학과별 탐색 표 (AI 협업)	3

모든 함수는 **입력 DataFrame을 바꾸지 않는다**

자기 점검

1. `info()`에서 결측을 찾는 방법은?
2. `loc`과 `iloc`의 차이는?
3. 학번을 `dtype=str`로 읽는 이유는?
4. `df[cond]["col"] = 값`이 원본을 바꾸지 못하는 이유는?
5. 결측을 0으로 채울 때와 평균으로 채울 때 결론이 달라질 수 있는 예는?

정리와 다음 주

오늘	핵심
구조	Series · DataFrame · 인덱스
탐색	head · info · describe · isna · value_counts
선택	`[]`, `loc`, `iloc`, `& | ~`, `isin`, `between`
가공	정렬, 새 열, `pd.cut`, `loc` 수정
AI	`and` → `&`, 체인 인덱싱 → `loc`, count vs size

다음 주 · Week 11 데이터 정제 · 변환 · 집계 — 지저분한 설문 데이터